Emergent Overlays: Autonomous Adaptation for Efficient Broadcasting in Heterogeneous Mobile Ad-Hoc Networks

Abstract*—Broadcast is a fundamental operation in Mobile Ad-Hoc Networks (MANETs). A large variety of broadcast protocols have been proposed to deal with network dynamics, maximize coverage and minimize the impact of collisions. These protocols follow either an unstructured approach based on the control of a flooding process or a more structured approach involving the creation of a dissemination overlay. The selection of the appropriate broadcast protocol for a MANET is a difficult task. Each protocol is better adapted to particular configurations of nodes, depending in particular on their density over space or their mobility. For heterogeneous node configurations, a single protocol is only adequate for some regions of the network offering poor performance elsewhere. This paper presents a novel adaptive approach to broadcast in heterogeneous MANETs. This approach allows MANET nodes to pick dynamically the most appropriate broadcast protocols for different regions of the network. Our approach ensure the interoperability between heterogeneous protocols with no impact on broadcast coverage.** Etienne ***►following sentences are probably giving too much detail and can be summarized◄ **It allows nodes in dense and relatively static zones to dynamically emerge overlays where they are beneficial, while the rest of the system uses more flexible unstructured approaches. Our policies, based on local and collective observation of the MANET state, allow nodes to collectively decide when to switch and aim at minimizing energy costs, while maximizing performance and coverage. Our evaluation results show that our adaptive approach leads to better performance and lower network consumption.**

I. Introduction

Etienne ►*proposed introduction outline*

- *MANETs are important due to advances in the IoT and the fact that not all systems will benefit from a fixed infrastructure (too costly, too seldom useful, etc.) [1]*
- *broadcast fundamental operation in MANETs, including for routing purposes, aggregation etc. [2], [3] - multiple protocols have been proposed for broadcast in MANETs*
- *broadcast in MANETs happens by the retransmission of a broadcast message by nodes until it reaches all nodes in the system. The objectives is to reach full coverage whilst spending as little per-node and overall energy as possible*
- *According to the survey by Ruiz and Bouvry [4], protocols can be classified in two main categories: protocols that use pre-established overlays, and those that do not use an established topology. This is similar to the typical unstructured vs structured overlays classification in P2P systems. Give a few details about the way unstructured approaches are based on controlled flooding (and therefore only use information available at the time of the propagation to decide on forwarding decisions) while approaches based on overlays can use information collected a priori and structure established also a priori.*
- *previous studies [5], [6] have shown that the performance and costs of each broadcast approach was intimately linked to environmental aspects of the MANET. In particular, density and mobility are key factors. The intuition is the denser the network is, the better it is to use an overlay to avoid collisions and speed up transmission (in less dense networks this is not as helpful and may lead to partitions if not up-to-date); and, conversely, the more mobility there is, the less useful an overlay is since overlays constructed in such context quickly become deprecated, posing risks of having partitions or bad coverage.*
- *choosing the appropriate protocol prior to the deployment of a MANET is difficult for a homogeneous network where the density and mobility patterns are not known a priori*
- *several MANETs are heterogeneous [7], [8], [9] and the density and mobility in different parts of the system will differ (find some convincing real life use cases to back this up, like the festival scenario)*
- *this makes the problem even more complex as the best algorithm for a part of the network may not be the best for another part.*
- *In this work, we are motivated by the notion that no one size fits all, and we are therefore interested in making broadcasting more adaptive and consequently more efficient and less costly for heterogeneous MANETs.*
- *our approach relies on a combination of interoperability and adaptation: (1) we allow different parts of the network to use different broadcast algorithms and ensure that coverage guarantees are maintained; (2) we propose dynamic adaptation mechanisms that allow nodes to dynamically select appropriate protocols based on the observation of the nodes deployment context. Our adaptation mechanisms range from simple approaches for a single node to coordinated approaches involving multiple nodes.*
- *We evaluate our approach using (include here highlights from the evaluation)*
- *Our contributions are (recap contributions)*
- *paper outline*

◄

II. Related work

Etienne ► *other papers to consider:*

- *[?] seems to use adaptation of parameters for VANET broadcast*
- *papers on hybrid unstructured and structured networks (at the end of the section)*
  - *Book chapter [16] provides a survey*
  - *[17] is a measurement study of unstruct. vs struct. systems – can be used as a generic pointer to the debate in the p2p community 10 years ago on the merits of the two approaches.*

Algorithm	Requirements								
	No need for One-hop neighbor	No need for Two-hop neighbor	No need for periodic control messages	Piggyback control messages	Use of thresholds	Stochastic decision	Source-independent	Allows Switch Protocol	Full coverage ($> 95\%$)
Hypergossiping [10]	×	✓	×	✓	✓	✓	✓	×	×
DPR [11]	×	×	✓	×	×	×	✓	×	×
Viswanath and Obraczka [12]	×	✓	×	×	✓	✓	×	×	×
Smart-Flooding [13]	×	✓	×	×	×	×	×	×	✓
Cartigny and Simplot [14]	×	✓	×	×	✓	×	×	×	×
Xu and Gerla [15]	×	×	×	✓	✓	✓	✓	×	✓
Our approach	✓	✓	✓	✓	✓	✓	✓	✓	✓

TABLE I

SURVEY OF EVALUATION CRITERIA USED IN DIFFERENT ADAPTIVE BROADCAST ALGORITHMS. **ETIENNE** ►*The table mixes several aspects that should probably be separated. I would recommend to have: (1) the characteristics of the protocol, e.g. does it use controlled flooding, an overlay, or both, does it combines two protocols, or adjusts the parameters for a protocol, or does both, is it autonomous or does it rely on some form of infrastructure, etc. (2) the mechanism used for the adaptation (mechanism = does it switch protocols, does it ensure interoperation, etc. and the policy = does it use threshold-based rules etc.). Then, (3) the type of information considered for the adaptation (most often, the density – there are no others?). And finally, (4) what is required to collect this information (these are the col. we already have here). I am not sure to see the interest of the "Piggyback control messages" column: can't all approaches that use periodic messages also embed information in broadcast messages instead? Finally it is indicated that we use a stochastic decision: I am not sure about this?*◄

– *Hybrid approaches in p2p networks are generally emerging a structure for efficiency amongst the stable peers, while leaving unstable or newly-joined peers use a more unstructured approach. Examples include HyPO [18] for video streaming where stable peers form a backbone tree and unstable ones are loosely connected in a mesh, or Flower-CDN [19] that uses a similar approach to build a CDN.*

◄

Etienne ►*general comments on the RW:*

- *it grows too long. I would suggest grouping papers by class, and not to discuss all the details of the implementation but rather emphasize the similarities. A paragraph per work is probably too much.*

◄

A wide range of routing protocols have been proposed for MANETs over the years, including **Yehia** ►*meta-examples*◄. Our focus is not particularly on a specific protocol, but rather on adaptation solutions that enable MANET nodes to switch between protocols to adapt to changes in their context. Some works have been done to facilitate such adaptation. For instance, there are protocols combining two approaches to broadcast (e.g. gossip and DTN). Another family of protocols adjust their parameters based on environmental observations, whilst other solutions employ hierarchical schemes or topology-aware schemes for reshaping their broadcasting approach. **Etienne** ►*it seems from the description of works below that some system actually do both (combine different protocols and adjust parameters).*◄ A summary of this related work is found in Table I where we identify the different mechanisms used by related work to identify the need for and actuate adaptation in MANETs. We now discuss these works.

Etienne ►*use titles like the following one to group works by classes.*◄

Adaptive-based broadcast algorithms. Hypergossiping [10] is a broadcast strategy protocol designed for MANETs where the authors combine two different protocols: Probabilistic Flooding [20] (known as gossiping in the paper) and Hyper-Flooding [21]. Hyper-flooding is a mechanism that forces each node to keep a track of its current neighbors. In case the same message is received again and new neighbors have been added between messages, then the node re-broadcasts the message. Hypergossiping was designed for partitioned networks where partitions of mobile nodes are constantly changing. Inside each partition, nodes use the Probabilistic Flooding protocol to send and receive messages adapting their local probability of forwarding incoming messages based on the local density of **Etienne** ►*around?*◄ the node. If two partition joins, the nodes which detect the join will re-broadcast the messages using Hyper-Flooding. Hypergossiping cannot work with partitions using different protocols and not **Etienne** ►*no?*◄ switch mechanism is developed.

Zhang et al. [11] propose an approach called Dynamic Probabilistic Route (DPR) discovery, based on a combination of probabilistic flooding and counter-based schemes. **Etienne** ►*warning: this must be defined before (in intro probably)*◄ Each node has a probability P to retransmit an incoming message depending on its local density. The protocol does not require periodic control messages in order to collect neighborhood information, but piggybacks this information on broadcast messages instead. The information piggybacked is used by the receiving nodes to determine their own local density, which will be used to determine whether the network is sparse or dense. If the network is deemed to be sparse, the node will always retransmit the message. However, if the node is in a dense network, P will be calculated based on the density of the

local neighbors as well as a cumulative amount of neighbor mobile nodes. If a node receives a message containing the same list of neighbors as its local list, then the node will not broadcast the message. As a result, the necessity know in advance the number of neighbors in order to calculate P adds some waiting time in estimating the local neighbor list. **Etienne** ►*revise last sentence*◄

Viswanath and Obraczka [12] present an algorithms that allow each node to switch among three different flooding modes: *plain*, *hyper*, and *scoped* flooding. Hyper-flooding is used to obtained the high reliability necessary for mobile scenarios with high mobility. It also has the highest overhead. In contrast, scoped flooding is used when nodes have a lower velocity, aiming to reduce redundant broadcast messages and obtaining a lower overhead. The nodes send periodic control messages to keep track of their one-hop neighbors. Each node uses two different thresholds to switch among these flooding mechanisms depending on the control messages previously received. Thresholds are modified based on the number of collisions. Their algorithm achieves a coverage between 86% and 90%, without a guarantee of full coverage. However, our proposal can adapt different kinds of broadcast algorithms, using the most suitable algorithm, such as counter-based or area-based among others, being more flexible for the requirements of diverse kind of situations.

Abdou et al. [13] propose an adaptive broadcasting protocol for MANETs called Smart-Flooding. This mechanism utilizes the Shadowing Pattern [22], a propagation model that is a realistic and probabilistic scheme based on outside monitored communications, together with an evolutionary algorithm. According to this Shadowing Pattern, each node modifies its own local parameters using the evolutionary algorithm and the network density calculated by sending periodically control messages.

Cartigny and Simplot [14] propose probabilistic algorithms for MANETs combining a simple probabilistic algorithm, a distance-based scheme [20], and a neighbor elimination scheme [23], [24] to reduce the number of broadcast messages sent through the network. In the distance-based scheme a node broadcasts a message only if the distance between itself and the receiver is larger than a specific threshold. In this work, periodically control messages are sent by the nodes in order to know the number of neighbors around themselves. The authors show that these algorithms do not achieve full coverage because the messages are broadcast to nodes that are in the border of the radio area and the probability of obtaining an optimal distance increases.

Xu and Gerla [15] describe a heterogeneous routing protocol based on a clustering scheme that allows different MANET broadcasting protocols to work together. The authors predefine specific nodes to be backbone nodes that can inter-communicate using long distance communications (non-backbone nodes do not). Each backbone node creates a cluster with a specific protocol used by the nodes contained in its area. The nodes, excluding backbone ones, can move among different backbone areas switching to the corresponding protocol. Therefore, when a broadcast message is sent, the backbone nodes send the broadcast message among them, and then among nodes in their respective cluster areas. Compared to our research, our approach achieves full broadcast coverage permitting the network being used by different broadcast algorithms and also defines a switching mechanism that allows nodes of different broadcasting algorithms to communicate among themselves. **Etienne** ►*I would compare with our approach by saying that we are more autonomous (nodes take the switching decisions autonomously) and that we do not require the support of an infrastructure.*◄

Yehia ►*synthesise and identify gaps in the above RW*◄

III. Overview

Etienne ►*write a short intro to section*◄

Algorithm 1: Network layer API

```
1 send(m: message) : void          // send to all nodes in
    wireless range
2 receive(m: message) : void  // callback when a message
    is received
```

Algorithm 2: Broadcast protocol API

```
1 init(pl: payload) : message          // create a new message
2 emit(m: message) : void       // initiate a new broadcast
3 handle(m: message) : void           // handle an incoming
    broadcast
```

A. System model

Etienne ►*todo: add references to Algorithm 1 and Algorithm 2 in the text (if necessary)*◄

We generalize the architecture of MANET applications as three main layers. First, the network implements a wireless protocol of communication such as IEEE 802.11, where nodes share a common channel with Carrier Sense Multiple Access (CSMA) and collision detection capabilities. This network layer provides an API with two communication primitives:

- send(). This function disseminates a message in the transmission area $A_{emitter}$ **Etienne** ►*it is incorrect to use math mode for 'emitter' in $A_{emitter}$. If we actually need this notation (not sure it is used in the algorithms?) then we should use $A_{emitter}$ (see latex source)*◄ of the emitter
- `receive()`. This method allows the reception of messages for every node located within $A_{emitter}$

For the sake of simplicity, in the literature $A_{emitter}$ is depicted as a circle of radius T_x, which is the length of nodes' transmission range, centered at the position of the emitter.

Second, the Broadcast layer implements a dissemination protocol where a source node sends a message to the entire network by letting nodes act as relays and forward incoming messages. We present in form of API the main operations required by any broadcast protocol:

- init(): Nodes use this method to create a new message with the payload to disseminate. The metadata associated

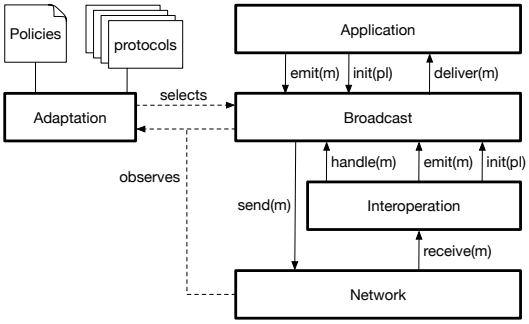

Fig. 1. Architecture of the system

with this payload differs between protocols. For instance, a location-based technique may add the emitter's geographic position obtained from a GPS device to the metadata.

- emit(): Source nodes use this method to initiate a broadcast session.
- handle(): This method is triggered on every message reception and implements one broadcast protocol.

The third main layer is the MANET application per se and it is placed on the top of the Network and Broadcast layers. When the application requires to disseminate the information I, a new broadcast message is created by calling init() with I as argument. The dissemination of I starts with a call to emit(), its propagation to others nodes in the network is performed by send() and any message in the interest of the application will be delivered via handle(). **Etienne** ►*false, this is with deliver(). handle() is from the network to the bcast protocol*◄

The architecture of a MANET application is depicted on Figure 1 along with two more layers: Interoperation and Adaptation. These two layers extend an application with our adaptive approach in a network of nodes with heterogeneous configurations, by monitoring the dynamics of the network and choosing the best broadcast protocol, accordingly. **Etienne** ►*revise previous sentence, it may seem that the* ◄

B. Framework of adaptation

We are interested in designing an *adaptive* approach to broadcast in MANETs. Adaptation refers to the ability of the system to change its behavior based on the observation of its deployment environment and performance. In our target context, adaptation refers to the ability to dynamically select and configure the protocol used for broadcasting. It is important to note that this adaptation and protocol selection should be possible not only for the entire system but also, just for a part of it. Thus, we must guarantee interoperability among heterogeneous regions of nodes. **Etienne** ►*the following tries to provide more details than is needed here, yet is unclear. This section should provide the high-level of the separation between mechanism and policy and not detail how each work.*◄ More concretely, every relay node between the source of a broadcast message and each destination node must perform a forward operation, such that there is any lost of messages when the emitter and receiver are not necessarily running the same protocol; our layer of interoperability deals with this regard.

1) Interoperation layer: **Etienne** ►*the content of this section is not in the right place. It is not the purpose of the overview section to present the details of how the interoperability and adaptation work. This is done in the two subsequent sections. Overall this section should be half a page, not more.*◄

We propose an interoperation mechanism between the Network and Broadcast layers, as shown in Figure 1. The basic idea is to broadcast a received message as a source node when the protocol of the emitter and the one form the receiver differ. **Vicent** ►*from?*◄ We assume that the protocol identifier is piggybacked on each broadcast message, note that this action does not modify the content of the message nor the metadata use by the protocols to their functioning. **Vicent** ►*"on each broadcast message. Note that this action does not modify the message content, nor the metadata used by ..."* ◄ This simple adaptation mechanism is depicted in Algorithm 6. The function `receive()` (line 4) acts as a wrapper for the communication primitive with the same name at the Network Layer. **Vicent** ►*Here Layer is in capital but before wasn't. What do you prefer?*◄ When the protocol identifiers of the emitter and receiver nodes differ, a new message will be forward **Vicent** ►*forwarded?*◄ keeping the original payload (line 8) but using the receiver's protocol instead (line 9). This algorithm have a drawback, though a message with the same payload is forwarded twice when the protocol identifiers of two emitters differ. A more efficient adaptation mechanism is shown in Algorithm 7 where we use timers to delay.... **Raziel** ►*TODO:*

- *finish with the description of Algorithm 7*
- *argue that these simple and yet efficient mechanism guarantees coverage for any protocol and mention the proof in section 4*
- *create Broadcast super class to reduce the space used by all implementation of broadcast*
- *give an overall overview of the Adaptation layer saying that its detail explanation requires another section*

◄ **Etienne** ►*none of these things belong in section 3.*◄

IV. Broadcast Interoperability

Etienne ►*this section discussed the mechanism for using different broadcast protocols and how they can interoperate*◄

Etienne ►*suggested outline*◄

- define the problem: we have a number of broadcast protocols P and nodes can use any of these protocols. Some protocols have only a reactive approach (i.e. their function is called only when an incoming message arrives from the network or when there is a request from the application), while others have a mix of a reactive approach together with a daemon that can initiate actions at predefined times (e.g. to send beacons and form an overlay)
- we consider that the network has a simple API (Algorithm 1) and that all broadcast protocols also use a common API (Algorithm 2).
- a message is initiated at any node and must reach all the others
- assume that any node does not know which protocol its neighbors are running
- if a node receives requests (e.g. for overlay construction) that are not for the locally-running protocol, they ignore them (this does not apply to messages carrying a payload)

Algorithm 3: Flooding-based protocols **Etienne** ▶*Several todos in this algorithm: (1) there are no deliver() calls to the application; (2) it is not possible to reach line 21 after the first reception due to line 15. (2) fixed timer based and random timer base are very close, they should be merged. (3) in the counter based case the message will never be delivered to the application as we will never reach line 25. It is not necessary to declare new variables such as t to hold a random value. In these algorithms, the retransmission is never canceled even if the same message is received several times. I thought this was the principle of the timer based algorithms. It is also not necessary to write "on timer expires" but to use a simple wait() command – this is pseudocode.*◀

Raziel ▶*implementation detail: in text mention that the set "delivered" must be cleaned periodically to avoid collecting an ∞ number of delivered messages.*◀

```
variables
    flooding_mode          // type of dissemination to perform
    delivered              // log of messages delivered to application
    use in counter-based version
        c []                                   // map of counters
        attributes of map
            c[m.id].receptions    // receptions of message m.id
        constants
            limit_of_receptions        // number of allowed receptions
    use in timer-based versions
        constants
            timeout                // default waiting time
function handle(m: message)
    if m.id ∉ delivered then
        switch flooding_mode do
            case SIMPLE do
                send(m)
                delivered ← delivered ∪ {m.id}
            case COUNTER-BASED do
                if c[m.id].receptions < limit_of_receptions then
                    send(m)
                    c[m.id].receptions ← c[m.id].receptions + 1
                else
                    delivered ← delivered ∪ {m.id}
            case FIXED_TIMER-BASED do
                delivered ← delivered ∪ {m.id}
                t ← newTimer(timeout)          // create new timer
                wait until t expires
                    send(m)
            case RANDOM_TIMER-BASED do
                delivered ← delivered ∪ {m.id}
                    // get a random number between 0 and timeout
                r ← newRandom([0, timeout])
                t ← newTimer(r)        // create new timer
                wait until t expires
                    send(m)
function init(pl: payload)
    m′ ← newFloodingMsg
    m′.payload ← pl
    return m′
function emit(m: message)
    delivered ← delivered ∪ {m.id}
    send(m)
```

Algorithm 4: Overlay-based broadcasting **Etienne** ▶*(1) no types and comments for variables? (2) delivery calls missing (3) line 18 should be outside the if on line 16 (4) lines 20–23 would fit in a single call (4) it is a daemon, not a demon (5) update is buggy: where is observed neig. created /maintained? (6) where is function applyOverlayRules()? This is the interesting one.*◀

```
variables
    nodeId
    am_i_relay
    delivered
    neighbors
    observed_neigs
    observed_relays
function handle(m: message)
    if m.id ∉ delivered then
        switch m.type do
            case CONTROL do
                observed_neigs ←
                  observed_neigs ∪ {m.emitter}
                if m.isRelay = TRUE then
                    observed_relays ←
                      observed_relays ∪ {m.emitter}
            case BROADCAST do
                if am_i_relay = TRUE then
                    send(m)
                    delivered ← delivered ∪ {m.id}
function demon()  // periodic daemon to send control messages
    ctrlMsg ← newControlMsg
    ctrlMsg.emitter ← nodeId
    ctrlMsg.isRelay ← am_i_relay
    send(ctrlMsg)
function update() // periodic update of relay status
    if neighbors ≠ observed_neigs then
        neighbors ← observed_neigs
        am_i_relay ← applyOverlayRule(neighbors)
        observed_neigs ← ∅
        observed_relays ← ∅
    else
        am_i_relay ← reduceBackbone(observed_relays)
```

- we want to ensure that the broadcast is complete
- show a naive example with an overlay based system and one based on flooding that if each node follows its own protocol only then we have risks of incomplete dissemination
- we detail and prove correct in this section an algorithm that ensures protocol interoperability, that is, the broadcast succeeds even if it crosses an arbitrary number of regions using different protocols.

  Etienne ▶*example protocols for broadcast*◀
- detail examples of broadcast protocols of the two classes
- Algorithm **??** for simple flooding
- **Etienne** ▶*also need to include others!*◀

  Etienne ▶*simple adaptation (act as source)*◀
- describe simple adaptation (Algorithm 6)
- prove it correct under the assumption that for nodes that are in range (no partition) and using the same protocol then we

Algorithm 5: Location-based broadcasting **Etienne** ►*(1) why do you consider only the first message? (line 8) (2) same as for the flooding algorithm, we never cancel a timed retransmission? But then what is the point of using locations if we eventually always flood???*◄

```
1  variables
2      delivered
3      t[]                          // map from message ids to timers
4      attributes of map
5          t[m.id].time                          // remaining time
6          t[m.id].payload                  // content of message
7  function handle(m: message)          // on message reception
8      if m.id ∉ delivered then
9          forward ← applyLocationRule(m.emitter.position)
10         if forward = TRUE then
11             if t[m.id] ≠ ⊥ then
12                 t[m.id] ← ⊥                        // cancel timer
13             m.emitter.position ← getPosition()
14             send(m)
15             delivered ← delivered ∪ {m.id}
16         else
17             t[m.id] ← newTimer              // create new timer
18             t[m.id].time ← getNewTimeout(m.emitter.position)
19             t[m.id].payload ← m.payload      // keep message
                payload
20             wait until t[m.id].time expires
21                 m.emitter.position ← getPosition()
22                 send(m)
23                 delivered ← delivered ∪ {m.id}
```

Algorithm 6: Simple adaptation mechanism

```
1  variables
2      p          // local instance of broadcast protocol
3      p.pid                          // protocol identifier
4  function receive(m: message)   // on message reception
5      if p.pid = m.pid then
6          p.handle(m)
7      else
8          m′ ← p.init(t[m.id].payload)
9          p.emit(m′)
```

guarantee that the connectivity is good. Use examples from previously-described algorithms to back this up (e.g. in the overlay-based one a node is not a relay only if it knows for sure that its neighbors are covered, by default, and in particular when the system starts, it is a relay). **Yehia** ►*the "by default" bit is confusing. reword to clarify.*◄

- show that while correct, it leads to broadcast storms at the frontiers (use a figure)

Etienne ►*better adaptation (using the timers)*◄

- describe use of timers (Algorithm 7) by relating them to the use of timers in the controlled flooding approach (description of broadcast above)
- prove that interoperability remains correct (using a proof by contradiction)

Algorithm 7: Efficient adaptation mechanism

```
1  constants
2      timeout                        // default waiting time
3  variables
4      p          // local instance of broadcast protocol
5      p.pid                          // protocol identifier
6      t[]                // map from message ids to timers
7      attributes of map
8          t[m.id].time                          // remaining time
9          t[m.id].payload                  // content of message
10 function receive(m: message)   // on message reception
11     if p.pid = m.pid then
12         if t[m.id] ≠ ⊥ then
13             t[m.id] ← ⊥                        // cancel timer
14         p.handle(m)
15     else
16         if t[m.id] = ⊥ then
17             t[m.id] ← newTimer          // create new timer
18             t[m.id].time ← timeout
19             t[m.id].payload ← m.payload      // keep message
                payload
20             wait until t[m.id].time expires
21                 m′ ← p.init(t[m.id].payload)
22                 p.emit(m′)
```

V. Adaptation Policies

Etienne ►*the policies need better framing: we must present first what is the control/feedback loop and say we will present a few variants. We must describe clearly what is the problem first. We could separate between the data available to the nodes. I insist that having a table linking the currently-running protocol and the available data would be very useful, as it drives the type of adaptation policy we can have.*◄

Etienne ►*the proposed strategies (algorithms) integrate the threshold-based rules directly in the algorithm. It would be clearer to put them separately as predicates provided to the policies, and that can be reused across multiple decision-making strategies (e.g. the simple local one vs the complex collective one).*◄

Etienne ►*when using adaptation policies, it is necessary to use hysteresis to avoid constant oscillations between two protocols. We can also include a grace period where a node keeps its protocol for a time before being allowed to switch again.*◄

In this section, we present the adaptation policies used in Section VI. As alternative of using periodic control messages, the adaptation policies proposed utilize a mechanism where each node saves the MAC address of the sender to its own local neighbor list each time a message is received, and it also attaches a timer to the new MAC added. When the node receives a message using a MAC that is already in its neighbor list, the timer associated to that MAC is restarted. If a MAC timer expires, the MAC will be deleted from the neighbor list.

As a result, when an adaptation policy request to a node its local density, the node will use its neighbor list to know how many nodes are in its neighborhood. As it has been demonstrated in [6], the node density is a significant factor

to decide which protocol is the most suitable. In this paper, we defined a threshold to conclude which protocol will be more adequate. In case of the node has more than 15 neighbor nodes, then the most suitable protocol to use will be flooding. Otherwise, it will be MPR. This local estimation density has been used for testing purposes, nevertheless, in future work, it might be redefined if necessary.

A. Local Policy

Local Policy is an adaptation policy where the decision to switch the protocol is made locally and individually, by each node with no collective decision. Each node checks periodically its local density using its neighbor list and, in case there is a more suitable protocol according to its local density, then the node will switch its protocol; independently of the protocols used by its neighbors. This adaptation policy is presented in Algorithm 9.

B. Share Will Switch Policy

Share Will Switch Policy (SWSP) is an adaptation policy based on a local decision and collective protocol switch presented in Algorithm 10. For this purpose, each node has a local Boolean variable called *willSwitch*. Thus, each node periodically checks its local estimated density and, in case another protocol is more suitable, *willSwitch* variable of this node is set to TRUE, adding to each message sent the *willSwitch* variable and the protocol the node aims to switch(*pSwitch*).

In case a node with *willSwitch* as TRUE receives a message containing *willSwitch* as TRUE and *pSwitch* in the message equals to *pSwitch* of the node, therefore, this node switches to *pSwitch* protocol and broadcasts a message attaching *willSwitch* as TRUE and *pSwitch*. After sending the message, the node will set the *willSwitch* as FALSE.

On the other hand, if a node with *willSwitch* as TRUE receives a message using the protocol this node aims to switch, in that case, the node will switch to the desire protocol and will send a message attaching *willSwitch* as TRUE and *pSwitch*. After sending the message, the node will set the *willSwitch* as FALSE.

VI. Evaluation

In Section IV, we have presented a mechanism to support the interoperability between nodes that use different dissemination algorithms. Since this mechanism modifies the behavior of each node, we can expect some performance impact in comparison to previous approaches. Likewise, our approach requires the use of adaptation policies as described in Section V we have described different policies. In this section, we evaluate our approach under realistic conditions. In particular, we are interested in finding answers to the following research questions:

RQ1 What is the cost of our adaptation mechanism in terms of energy consumption, broadcast latency, and coverage?

RQ2 Is it truly beneficial to use an adaptive approach instead of previous dissemination algorithms?

Algorithm 8: Collective enactment of an elasticity rule (disposal of data structures omitted for clarity).

```
1  policy-specific functions and parameters
2      propose condition            // trigger for proposing adaptation
3      accept condition             // trigger for accepting adaptation
4      participants threshold       // nodes required to enact adaptation
5  constants
6      B = bit field size
7      F = field of B bits, all false except one random bit set to true
8  variables
9      p        // local instance of broadcast protocol
10     proposed adaptation: map (proposition id, proposition) init ∅
11     ongoing decisions: map (proposition id, proposition) init ∅
12 function func triggerparams          // on propose condition trigger
13     adp = new adaptation proposition
14     proposed adaptation.add (adp.id,adp)
15     m ← p.init(adp)
16     m.id = random identifier
17     m.class = proposal
18     p.emit(m)
19 function func receivemessage m       // upon reception of collective enactment message
20     switch m.class do                // check message type
21         case proposal do             // adaptation proposal
22             if accept condition(proposal) then
23                 ongoing decisions . add (m.id,m.pl)
24                 return (true)
25             else
26                 ref = new refuse message
27                 m′ ← p.init(ref)
28                 m′.id = m.id
29                 m′.class = refuse
30                 return(m′)
31         case refuse do               // refuse notification
32             if ongoing decisions[m.id] then
33                 agg = new aggregation message
34                 agg.F = F
35                 m′ ← p.init(agg)
36                 m′.id = m.id
37                 m′.class = aggregate
38                 return(m′)
39             else
40                 return(false)
41         case aggregation do          // ongoing aggregation
42             if ongoing decisions[m.id] then
43                 if ! ongoing decisions[m.id].F then
44                     ongoing decisions[m.id].F = F
45                     ongoing decisions[m.id].timer = new timer(agg waiting time)
46                 ongoing decisions[m.id].F = ongoing decisions[m.id].F ∪ m.pl.F
47             return (false)
48         case enact do                                  // enactment
49             if ongoing decisions[m.id] then
50                 apply protocol change
51                 return (true)
52             return(false)
53 function expires(ongoing decisions[id]: timer)
   // aggregation timer expires
54     if proposed adaptation[id] then
55         expected = t · (1 − (1 − 1/B)^t)
56         if number of bits set to 1 in ongoing decisions[id].F ≥ expected then
57             enact = new enactment message
58             m′ ← p.init(enact)
59             m′.class = enact
60             p.send(m′)
61         else
62             return(false)
```

Algorithm 9: Local Policy

```
1  function trigger(localTimer)                    // on propose check
   adaptation trigger
2      if Density ¿= 15 then
3          if p.pid ≠ flooding then
4              switch (flooding)
5      else
6          if p.pid ≠ MPR then
7              switch (MPR)
```

Algorithm 10: Share Will Switch Policy

```
1  variables
2      willSwitch      // boolean of willingness to switch
3      pSwitch     // broadcast protocol that node wants
         to switch
4  function receive(m: message)          // Modification of
   message reception
5      if m.willSwitch = TRUE then
6          if m.pSwitch = pSwitch then
7              switch (pSwitch)
8              willSwitch = FALSE
9          else
10             if m.pid = pSwitch then
11                 switch (pSwitch)
12                 willSwitch = FALSE
13     Continuing received mechanism
14 function trigger(MACTimer)                   // on propose check
   adaptation trigger
15     if Density ¿= 15 then
16         if p.pid ≠ flooding then
17             willSwitch = TRUE
18             pSwitch = flooding
19     else
20         if p.pid ≠ MPR then
21             willSwitch = TRUE
22             pSwitch = MPR
```

RQ3 How do different adaptation policies affect the performance of our adaptive approach?

RQ4 What adaptation policy better captures the dynamicity of a network topology?

To answer these questions, we perform controlled experiments using a network simulator. In the rest of this section, we first present our simulation environment, present and discuss the results of our experiments, and finally draw conclusions to answer the above research questions.

A. Simulation Environment

Our experiments are inspired by the scenario of a crowd of users carrying portable communication devices (e.g., smartphones) in an area with no infrastructure connectivity. In this area, there are several points of interest where people tend to stay. As a result, the density of devices in the points of interest is high, and the mobility is low. Meanwhile, fewer nodes constantly move in the area between point of interests. The dissemination of information from one individual to all the others is critical, and uses ad-hoc connections between devices. Our evaluation of broadcast algorithms for MANETs is mainly driven by two factors: *network density* and *nodes mobility*. We describe in the following the parameters used in our experiments.

a) Simulator.: We rely on OMNET++4.6/INET3.3, a simulation framework for the evaluation of networking algorithms [25]. This tool provides a full implementation of the IEEE 802.11g standard. In addition, it contains models for the mobility of nodes and for energy consumption.

b) Energy Model.: The power consumed by a node mainly depends on the wireless communication chip it uses. We selected the Broadcom BCM4329 [26], a commercial chip commonly found in modern mobile devices and which supports Wi-Fi 802.11g on the 2.4 GHz frequency band. As described in Table II, the energy consumed by the chip varies from only 648 nW in sleep mode to up to 1.206 W when the transmitter is active.

Mode	Consumption
Sleep	648 nW
Receiver Listen	244.8 mW
Receiver Active (RX)	295.2 mW
Transmitter Active (TX)	1.206 W

TABLE II
BROADCOM BCM4329 POWER CONSUMPTION MODES

To simulate this chip, we use the *StateBasedEnergyConsumer* model provided by INET. Laurent ►*ref in footnote?*◄ This model measures how many Joules are consumed based on the time spent by the chip on each consumption mode.

c) Communication Scenario.: Simulated participants use their devices to broadcast messages without external coordination and may attempt to do so at nearly the same time. We set our experiments to perform 200 broadcast sessions with a frequency between broadcasts of 0.1s. During a broadcast session, a randomly selected node acts as a source and broadcasts a randomly generated message. Each of the 200 messages is 32 bytes long and contains a unique identifier. Moreover, we assume normal wireless communications range and no environmental interference. The only reason to lost packets in the medium is due to collisions, which are handle by CSMA/CA protocol.

d) Simulation time.: As mentioned in Section IV, some algorithms need to build a topology. To guarantee that the first broadcast message is properly handled by all protocols, we empirically set an initial warm-up period of 2s, allowing CDS-based algorithms to perform this initial construction. Similarly, we wait 2s after the last broadcast session. The overall simulation time is therefore of 24 seconds.

e) Mobility Model.: We use a mobility model with points of interest (PoI) proposed by Piorkowski et al. [27]. In this model, all nodes move in short steps that are defined using two Gaussian distributions. To do so, nodes are classified in two groups: nodes in the vecinity of a PoI use a distribution $\mathcal{N}(0, \sigma_1^2)$, and nodes in between PoIs use $\mathcal{N}(0, \sigma_2^2)$ where $\sigma_1 < \sigma_2$. As a consequence, nodes in the vecinity of a PoI move slower than nodes in between PoIs. They also stay most

of the time in a PoI making the density in PoI higher than elsewhere.

To generate the traces, nodes are initially uniformly distributed on a map of 250 x 250 m. During each step of the simulation, nodes move according to the group in which they belong to. Table III summarizes the configuration parameters we used.

Parameter	Value
Number of nodes	500
Number of PoIs	4
Size of the map	250m x 250m
Radius that defines vecinity of PoI	25m
σ_1^2	0.1
σ_2^2	0.8

TABLE III
PARAMETERS OF THE MOBILITY MODEL.

Although we check that the initial network is connected, we cannot guarantee that all nodes are reachable for the whole duration of the experiment. We argue that it does not affect the experiments because all protocols are evaluated under the same conditions.

f) Algorithm-specific parameters.: The behavior of each protocol depends on its parameters, for which we favor values previously reported in the literature. For instance, we use 1.5s as the *hello time* for *MPR*. In the case of *ABBA*, we use 60 milleseconds as the maximum waiting time.

For adaptive approaches, we only allow switching between two protocols. At the beginning of the simulation, each nod is executing *simple flooding*. Afterwards, a node is free t change its protocol based on some policy. However, it can onl change back and forth between *simple flooding* and *MPR*. W are selecting these two algorithms based on previous studie that show their benefit in the scenarios we are considering [6

B. Evaluation metrics

To properly compare our approach against previous solu tions, we present the results of our experiments in terms of set of well-established metrics:

g) Coverage.: The percentage of nodes that successfully receive a broadcast message at least once. Since we are performing several broadcast sessions, we are interested in the evolution of this metric over time.

h) Saved Rebroadcasts (SRE).: The number of forwarding avoided by using a given algorithm compared to *simple flooding*. To compute the SRE of one algorithm A in a network of N nodes, we measure the number R_m of retransmissions made by A to disseminate a message m. The SRE_m for message m is then $N - R_m$. Since in our experiments we are disseminating many messages, we compute the mean *SRE* and use it as metric for our comparison study. This metric is useful to assess the performance of algorithms in terms of both network and energy usage.

i) Broadcast latency.: The time to disseminate a message with a coverage of 100%. The broadcast time helps to understand which algorithm to use with an application. For instance, some applications tolerate high-latencies while others need fast responses.

j) Energy consumption.: The energy consumed by nodes for the broadcast of all messages and, when applicable, for the construction and maintenance of the overlay.

C. Cost of adaptation mechanisms

In this Section, we measure the cost of the mechanisms proposed in Section IV to guarantee message dissemination across different base protocols. In particular, we evaluate the cost in terms of power consumption and broadcast latency for the mechanisms presented in Algorithm 6, Algorithm 7 and Algorithm **??**.

To perform this experiments we use a simple network topology where there are 500 nodes and one PoI. Half the nodes are located in the squared PoI (center of the map). In this PoI, the average node density is 40 while it is 5 elsewhere. Hence, we deploy *MPR* in the PoI and *simple flooding* outside. In the experiments, no node is moving and the source of every message is a node executing *simple flooding*.

Figure Figure 2 shows the power consumption for the three algorithms. As expected, Algorithm 6 is worst in this scenario because no optimization is applied. On the other hand, Algorithm 7 improves this metric. As previously discussed, we need to use Algorithm **??** to guarantee coverage; these experiments show that doing so comes at the cost of increasing energy consumption.

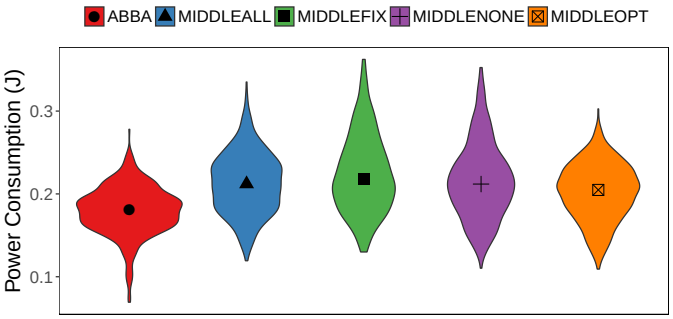

Fig. 2. Distribution of Power consumption per glue mechanism variant

In Figure 3, the *broadcast latency* is depicted. Observe that Algorithm 6 generates no overhead; messages are delivered under 100 ms. Likewise, Algorithm **??** does not impact the latency; hence, in principle we guarantee coverage without impacting the latency. Finally, the optimization presented in Algorithm 7 does negatively affect the latency.

In Figure 4, the *saved rebroadcast message* for the glue algorithm is shown.

Inti ►*You may be wondering where are the plots. I am generating them right now*◄

D. Comparison with base protocols

Figure 5 shows the coverage obtained when different algorithms are used to disseminate messages in a MANET. In

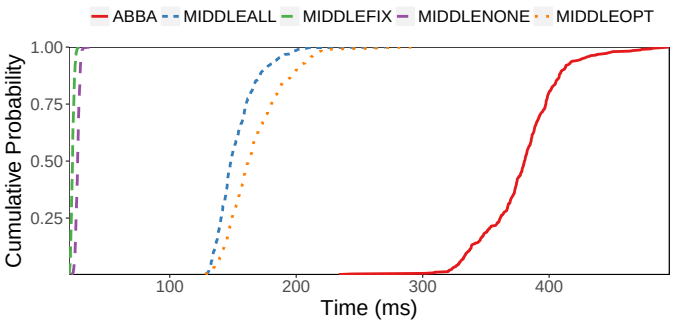

Fig. 3. Broadcast Latency per glue mechanism

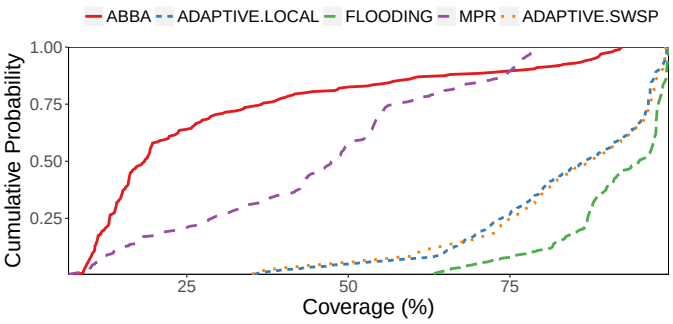

Fig. 5. Coverage per algorithm

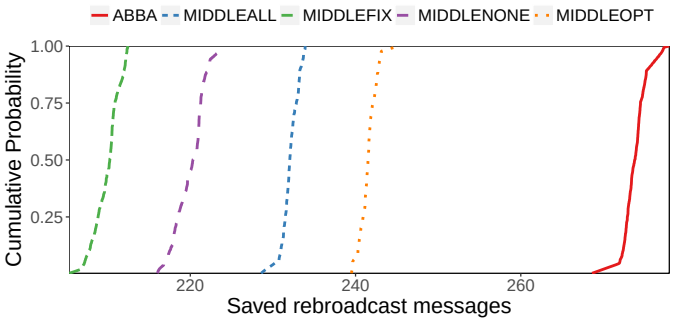

Fig. 4. Saved rebroadcast messages per glue mechanism

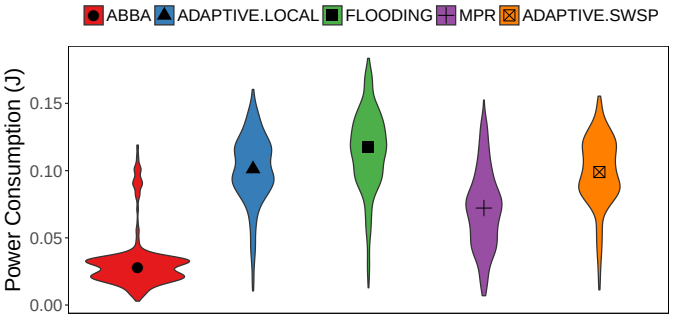

Fig. 6. Distribution of Power consumption per algorithm

particular, it depicts the results for previous approaches such as *ABBA*, *MPR*, *simple flooding* and our adaptive solution with its two policy variants. The first thing to notice is that the studied network is often disconnected as coverage is less than 100% for many broadcast sessions; **Yehia** ▶*median coverage is xx% across all sessions*◀. The second interesting point is that both our adaptive solutions significantly outperform both *ABBA* and *MPR* in terms of achieved coverage. Specifically, our *local* and *SWSP* policies achieve mean coverage of 83.2% and 83.4%, respectively, which closely follows the trend of the maximalist *flooding* protocol (91.2%). In contrast, *ABBA* only manages 29.4% mean coverage and *MPR* 45.2%. Moreover, the minimum coverage achieved by our adaptive solution is 35.4%, whilst this is considerably lower for the other protocols: 7.9% for *ABBA* and 6.7% for *MPR*. There are, however, instances where the pattern does not hold and our adaptive solution lags behind *flooding*. This happens when some nodes switch from *flooding* to *MPR* but *MPR* is still warming up, so the node takes some time before acting a relay when it should be doing so.

In terms of the differences between the particular adaptation policy adopted, we find little effect on coverage. Although in a few broadcast sessions the *local* policy behaves better than *SWSP*, the difference is negligible.

The power consumption for the same experiments is depicted in Figure 6. Although *ABBA* seems to outperform other approaches, it is worth noticing that its consumption is remarkably lower than that of others because it has the worst coverage, and also due to the fact that we are not measuring the cost of using a GPS as required by the original protocol. Previous work has shown that such power cost is non-trivial. Using the figures reported in a previous study [28], GPS would consume about an average of 2.83J per node. As for *MPR*, nodes using this protocol show a median power consumption of 0.072J. The difference of *MPR* with respect to our approaches is particularly pronounced because *MPR* is right-skewed (skewness of 0.21) while our approaches are left-skewed (at least -0.57). However, as we already discussed it coverage is less than in other mechanisms. Our adaptive approaches exhibit similar power consumption: median 0.101J for *local* and 0.099J for *SWSP*. The latter should be slightly higher due to the few additional messages sent from nodes using *flooding* to those running *MPR*, but in our experiments we could not observe this effect. **Yehia** ▶*why? needs explaining or remove comment*◀ This can be attributed to a better selection of what nodes switch their protocol. **Yehia** ▶*how can we be sure about this?*◀ Finally, *flooding* consumes more energy than other approaches (with a median of 0.118J, ≈18% more than our approaches) because nodes are continuously retransmitting messages. In summary, our adaptive mechanism and adaptive policies are able to behave similarly or better than other approaches in terms of power consumption but with significantly better coverage.

Finally, we discuss *broadcast latency*, another important

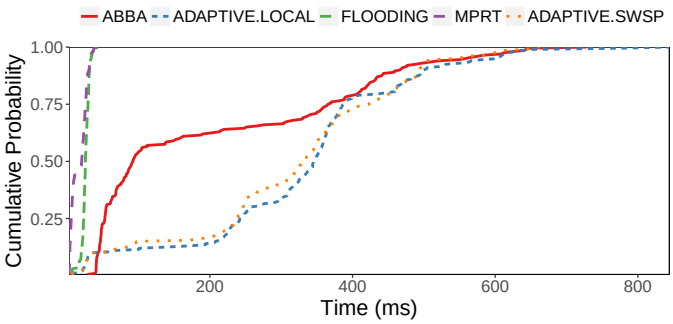

Fig. 7. Broadcast Latency per algorithm

metric in broadcast protocols. In Figure 7, we show the *CDF* of this metric for different algorithms. As can be seen, *MPR* and *flooding* have a very short *latency* as expected from their design. In particular, all messages are disseminated in less than 100 ms. *ABBA*, on the other hand, requires more time to deliver the messages. Although over 75% of the messages are delivered in under 400 ms, there is a long tail showing that some of them can take as long as 800 ms. The behavior of the adaptive approaches is similar to that of *ABBA*. This is indeed a poor performance; it is the consequence of the optimization discussed in Algorithm 7 where nodes wait before retransmiting a message coming from a foreign protocol. Notice that over 75% of the messages are broadcast in less than half a second, like in using *ABBA*. Also worth noticing is that there are no differences between the two adaptation policies.

E. Discussion

We now reflect back on the research questions posed at the beginning of this section in light of the experiment results.

The experiments in Section VI-C answer the research question *RQ1*. **Inti** ►*Say something smart here*◄

Using the results shown in Section VI-D, we can answer *RQ2* and *RQ3*. Concretely, using our adaptive approaches instead of previous solutions can be beneficial in deployment scenarios with PoIs where overall coverage is significantly improved (**Yehia** ►*two fold?*◄). *Flooding* is the only protocol offering better coverage, albeit only slightly, than our solution. However, it consumes more energy. On the other hand, the *ABBA* and *MPR* protocols offer lower power consumption but at the cost of notably diminished coverage.

Unfortunately, we have not found a conclusive answer to research question *RQ3*. That is, we cannot observe significant differences in the two adaptation policies we compare within the extent of our experiments. For instance, *SWSP* has a slight advantage in terms of energy consumption, but the difference is negligible. Meanwhile, the *Local* adaptation policy is marginally better in terms of *broadcast latency*. More extensive experiments are required, probably including other policies.

In summary, previous protocols such as *ABBA* and *MPR* are not able to cope with networks where node density varies within the network. This, perhaps, is unsurprising as they have been designed for homogeneous networks. As such, the coverage they offer is severely affected by natural topological factors that induce variance in density, which in turn affects mobility, such as the presence of PoIs. Strictly speaking, *flooding* is able to handle such variance due to its uncompromising maximalist approach. This, obviously, comes at the expense of power consumption.

It is worth noticing that the validity of these results is limited to the scenarios described in Section VI-A. That is, scenarios where there are several PoIs in an area, people tend to stay in the PoIs, and only a few move between PoIs. **Yehia** ►*i would actually explore this further, if possible. this will also allow us to have closure on RQ3*◄

VII. Conclusion

References

[1] P. Bellavista, G. Cardone, A. Corradi, and L. Foschini, "Convergence of MANET and WSN in IoT urban scenarios," *IEEE Sensors Journal*, vol. 13, no. 10, pp. 3558–3567, Oct 2013.

[2] B. Williams and T. Camp, "Comparison of broadcasting techniques for mobile ad hoc networks," in *3rd ACM International Symposium on Mobile Ad Hoc Networking & Computing*, ser. MobiHoc. New York, NY, USA: ACM, 2002, pp. 194–205. [Online]. Available: http://doi.acm.org/10.1145/513800.513825

[3] S. Basagni, M. Conti, S. Giordano, and I. Stojmenovic, *Mobile ad hoc networking*. John Wiley & Sons, 2004.

[4] P. Ruiz and P. Bouvry, "Survey on broadcast algorithms for mobile ad hoc networks," *ACM Computing Surveys*, vol. 48, no. 1, pp. 1–35, 2015. [Online]. Available: http://dl.acm.org/citation.cfm?id=2808687.2786005

[5] N. Aschenbruck, E. Gerhards-Padilla, and P. Martini, "Modeling mobility in disaster area scenarios," *Performance Evaluation*, vol. 66, no. 12, pp. 773 – 790, 2009. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S0166531609001072

[6] R. Carvajal Gómez, I. Gonzalez-Herrera, Y.-D. Bromberg, L. Réveillère, and E. Rivière, "Density and mobility-driven evaluation of broadcast algorithms for manets," in *37th Annual IEEE International Conference on Distributed Computing Systems*, ser. ICDCS, 2017.

[7] T. Camp, J. Boleng, and V. Davies, "A survey of mobility models for ad hoc network research," *Wireless Communications and Mobile Computing*, vol. 2, no. 5, pp. 483–502, 2002. [Online]. Available: http://dx.doi.org/10.1002/wcm.72

[8] F. Bai and A. Helmy, "A survey of mobility models," *Wireless Adhoc Networks*, vol. 206, p. 147, Jun. 2004.

[9] R. Ramdhany and G. Coulson, "Manetkit: A framework for manet routing protocols," in *28th International Conference on Distributed Computing Systems Workshops*, June 2008, pp. 261–266.

[10] A. Khelil, P. J. Marrón, C. Becker, and K. Rothermel, "Hypergossiping: A Generalized Broadcast Strategy for MANETs," in *Ad Hoc Networks Journal*. Elsevier, August 2006, pp. 142–153. [Online]. Available: http://www2.informatik.uni-stuttgart.de/cgi-bin/NCSTRL/NCSTRL_view.pl?id=INPROC-2006-42&engl=1

[11] Q. Zhang and D. P. Agrawal, "Dynamic probabilistic broadcasting in manets," *Journal of parallel and Distributed Computing*, vol. 65, no. 2, pp. 220–233, 2005.

[12] K. Viswanath, K. Viswanath, and K. Obraczka, "An adaptive approach to group communications in multi hop ad hoc networks," in *7th International Symposium on Computers and Communications*, ser. ISCC, 2002, pp. 559–566.

[13] W. Abdou, C. Bloch, D. Charlet, D. Dhoutaut, and F. Spies, "Designing smart adaptive flooding in MANET using evolutionary algorithm," in *4th International Mobile Wireless Middleware, Operating Systems, and Applications*. Springer Berlin Heidelberg, 2012, pp. 71–84.

[14] J. Cartigny and D. Simplot, "Border node retransmission based probabilistic broadcast protocols in ad-hoc networks," *Telecommun. Syst.*, vol. 22, no. 1-4, pp. 189–204, Jan. 2003.

[15] K. Xu and M. Gerla, "A heterogeneous routing protocol based on a new stable clustering scheme," in *IEEE Military Communications Conference*, ser. MILCOM, vol. 2, Oct 2002, pp. 838–843 vol.2.
[16] J. Leitão, N. A. Carvalho, J. Pereira, R. Oliveira, and L. Rodrigues, *On Adding Structure to Unstructured Overlay Networks*. Boston, MA: Springer US, 2010, pp. 327–365. [Online]. Available: http://dx.doi.org/10.1007/978-0-387-09751-0_13
[17] Y. Qiao and F. E. Bustamante, "Structured and unstructured overlays under the microscope: A measurement-based view of two p2p systems that people use," in *USENIX Annual Technical Conference*, ser. ATC, 2006. [Online]. Available: http://dl.acm.org/citation.cfm?id=1267359.1267390
[18] H. Byun and M. Lee, "Hypo: A peer-to-peer based hybrid overlay structure," in *11th International Conference on Advanced Communication Technology*, ser. ICACT, 2009.
[19] M. El Dick, E. Pacitti, and B. Kemme, "Flower-cdn: A hybrid p2p overlay for efficient query processing in cdn," in *12th International Conference on Extending Database Technology*, ser. EDBT, 2009.
[20] S.-Y. Ni, Y.-C. Tseng, Y.-S. Chen, and J.-P. Sheu, "The broadcast storm problem in a mobile ad hoc network," in *5th Annual ACM/IEEE International Conference on Mobile Computing and Networking*, ser. MobiCom. New York, NY, USA: ACM, 1999, pp. 151–162.
[21] Z. Cheng and W. B. Heinzelman, "Flooding strategy for target discovery in wireless networks," *Wireless Networks*, vol. 11, no. 5, pp. 607–618, 2005.
[22] D. Dhoutaut, A. Régis, and F. Spies, "Impact of radio propagation models in vehicular ad hoc networks simulations," in *3rd International Workshop on Vehicular Ad Hoc Networks*, ser. VANET. New York, NY, USA: ACM, 2006, pp. 40–49.
[23] W. Peng and X.-C. Lu, "On the reduction of broadcast redundancy in mobile ad hoc networks," in *1st Annual Workshop on Mobile and Ad Hoc Networking and Computing*, ser. MobiHOC, 2000, pp. 129–130.
[24] I. Stojmenovic, M. Seddigh, and J. Zunic, "Dominating sets and neighbor elimination-based broadcasting algorithms in wireless networks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 1, pp. 14–25, jan 2002.
[25] A. Varga and R. Hornig, "An overview of the OMNeT++ simulation environment," in *1st International Conference on Simulation Tools and Techniques for Communications, Networks and Systems*, ser. Simutools, 2008.
[26] BROADCOM. (2016, September) BCM4329 datashet. [Online]. Available: http://www.cypress.com/file/298626/download
[27] M. Piorkowski, N. Sarafijanovic-Djukic, and M. Grossglauser, "A parsimonious model of mobile partitioned networks with clustering," in *1st International Communication Systems and Networks Workshops*, ser. COMSNETS, 2009.
[28] A. Carroll and G. Heiser, "An analysis of power consumption in a smartphone," in *Proceedings of the 2010 USENIX Conference on USENIX Annual Technical Conference*, ser. USENIXATC'10. Berkeley, CA, USA: USENIX Association, 2010, pp. 21–21. [Online]. Available: http://dl.acm.org/citation.cfm?id=1855840.1855861
[29] P. Ruiz and P. Bouvry, "Survey on broadcast algorithms for mobile ad hoc networks," *ACM Comput. Surv.*, vol. 48, no. 1, pp. 8:1–8:35, Jul. 2015. [Online]. Available: http://doi.acm.org/10.1145/2786005
[30] K. Akkaya and M. Younis, "A survey on routing protocols for wireless sensor networks," *Ad Hoc Networks*, vol. 3, no. 3, pp. 325 – 349, 2005.
[31] I. . W. Group *et al.*, "Wireless lan medium access control (mac) and physical layer (phy) specifications," 1997.

APPENDIX

Broadcast is a fundamental operation in Mobile Ad-Hoc Networks (MANETs) [29]. Besides allowing to dispatch a message to all nodes, broadcast is often used as the basis of routing mechanisms [30]. A large variety of broadcast protocols have been proposed. In a recent survey [29], Ruiz *et al.* propose a taxonomy for classifying broadcast protocols. We focus on the two main groups of protocols identified in this taxonomy. Protocols of the first group (called "No Underlying Topology" in [29]) use a form of *controlled flooding*. Nodes decide independently *if* and *when* they must forward broadcast messages to the nodes in range of their wireless communication. This decision is made independently, for each new incoming message. The forwarding decision and the forwarding itself are therefore coupled. Protocols of the second group (called "Underlying Topology" in [29]) use a pre-constructed *overlay*. Nodes decide *if* they must forward incoming messages based on their role in this overlay. Typically, the overlay distinguishes between relay nodes that forward all incoming messages and passive receiver nodes that do not relay incoming messages. The construction of the overlay is independent from broadcast operations. This decouples the forwarding decision and the forwarding itself.

Selecting the most appropriate broadcast protocol for a given MANET deployment is a complex task. Furthermore, deployment conditions may change over time, hindering this decision-making. Deployment conditions for a MANET include the characteristics of individual nodes, such as their communication range and battery capacity, but also collective dynamics such as mobility and density in space. A comparison study [6] shows that the best protocol for a given situation, is generally not appropriate for a different situation. A typical example is that for a high density network where nodes are mostly static. In this setting, a protocol of the second class building an overlay is more efficient and robust. However, for a sparse network with highly mobile nodes, protocols of the first class based on controlled flooding typically perform better.

We are interested in designing an *adaptive* approach to broadcast in MANETs. Adaptation refers to the ability of the system to change its behavior based on the observation of its deployment environment and of its performance. In our target context, adaptation refers to the ability to dynamically select and configure the protocol used for broadcast. It is important to note that this adaptation and protocol selection should be possible not only for the entire system but also, and more interestingly, just for a part of it. Indeed, the deployment conditions may not be homogeneous for all nodes in a MANET . Some nodes may for instance be in densely-populated areas while other may be in sparsely-populated areas. Nodes in some region may be very mobile while nodes in another region may be mostly static. As a result of this heterogeneity, the best protocol for the nodes in one region may not be the best for the nodes in another region. Therefore, nodes in one region should be able to autonomously discover the need for local adaptation and perform a collective decision towards the switch to a different protocol. When switching from a protocol of the first group to a protocol of the second group, a local overlay will be created amongst the nodes of the group, leading to the notion of *emergent overlay*. Reversely, the switch from a protocol of the second group to a protocol of the first will lead to the disposal of the previously-constructed overlay. The existence of sub-systems using a different protocol than the rest of the system requires interoperability mechanisms, acting at the boundary of the systems using different protocols and ensuring continuity of the broadcast operation. MANETs are by nature decentralized: there is no global knowledge or centralized coordination between nodes. The adaptation process must therefore be autonomous and rely on local

interactions between nodes. The decision of switching to a new protocol will depend on observation made by nodes about the running protocol and their environment. The nature of the information that can be collected depends on the protocol(s) already running, and on the messages that these protocols use for their operation. The decision itself must rely on an autonomous process, whereby nodes collectively detect and enforce the boundaries of a sub-system and implement the protocol change for that sub-system.

We break down the problem of providing autonomous adaptation to broadcast in MANETs as follows:

- **Information collection.** The adaptation is based on rules that trigger based on collected information about the environment of nodes, and the performance of the currently-running protocol. Information is first available directly and locally at the nodes themselves, such as the level of remaining energy or, when turned on and used by the protocol, a GPS coordinate. Second, it is possible to collect information about the network, such as the amount of messages collisions happening due to concurrent sending by nodes with a common coverage. Nodes may for instance fetch the number of successful acknowledge messages from their local CSMA/CA (carrier sense multiple access with collision avoidance) [31] transceiver. Third, information can be inferred from the running protocol and broadcast operations. Protocols of the second group (using an overlay) typically require the exchange of neighbors lists between nodes, or even estimation of the geographical positions of these neighbors. This information can be used in addition to network-level information to infer the local density of the network. Collecting information that is already used by the protocol induces no additional costs in principle. In other cases, information can be obtained by piggybacking information on existing messages, which has a relatively low cost. However, not all type of information is available directly from any running protocol, or can be obtained by piggybacking on existing messages. Obtaining the list of neighborhoods when running a protocol of the first group would likely require the creation of additional messages, which would somehow defeat the purpose of using a protocol of the first group. The first task is therefore to determine, for each available protocol, what are the information types that can be collected and what are the types that cannot. The adaptation rules should be based solely on information that can be inferred at a low cost. It is also important to evaluate the cost of the observation and determine its impact on the actual broadcast and on the energy consumption of the system.
- **Adaptation rules definition.** The decision of switching from one protocol to the other is based on pre-defined *adaptation rules*. These rules are typically based on thresholds applying to the values of the observed information. As the information available will differ depending on the currently-running protocol, there can not be a single, universal adaptation rule that can be triggered independently of the currently-running protocol. Instead, rules should adapt to the context in which they will be evaluated and be able to cope with information of different qualities. For instance, if the exact location and identity of a nodes neighbors can be inferred from the currently-running protocol, the measure of local density will be more accurate than in a situation where it can only be estimated based on the amount of collisions seen at the MAC layer. Rules must therefore use thresholds that adapt to the quality of the information.
- **Collective and autonomous decision-making.** There is no centralized knowledge of control over a MANET. We need to design a mechanism by which a set of nodes collectively and autonomously decide that (1) they form a coherent whole for which adaptation is necessary and (2) they decide collectively on the new protocol that they are going to use. This decision should ensure a series of properties:
  - The selected sub-system is of sufficient size to make the adaptation decision worth it; we do not want a pair of nodes to decide that they are a new system, and preferably want a threshold on the minimal number of nodes that makes it worth adapting.
  - There should be no "holes" in the selected sub-system, that would contain an isolated part of the global system with only a few nodes. For instance, if a group of 100 nodes decide to switch to a protocol of the second group and emerge an overlay, we wish to avoid a isolated group of 5 nodes in the middle that keep using a protocol of the first group.
  - There should be a continuity of service: when an adaptation decision is made, the existing broadcast should terminate correctly (reaching at least the same coverage as the original protocol).
  - The nodes in a group should reach a consensus on the adaptation decision they make.
- **Interoperability between protocols.** Broadcast should cover the entire network including the sub-systems that have decided to adapt and switch to another protocol. At the boundary of a system that has decided to adopt a new protocol, messages will be received from one protocol and sent following another protocol. Our goal is to avoid any loss of coverage and to minimize the cost of such a barrier crossing. The naive solution, where any node receiving a message from a protocol it does not use as a new source for the same message, leads to local broadcast storms at the boundary of the sub-system. This should be avoided by designing mechanisms that guarantee the continuity of service between protocols while minimizing the cost of interoperability.